Guía de Sintaxis y Patrones: Spring Boot, JPA & React

Documento de Referencia Técnica y Preparación de Examen (ControlCheck 2)

BACKEND & FRONTEND

1. ELEMENTOS DE CÓDIGO EN BACKEND (SPRING BOOT & JPA)

1.1. Controladores REST y Manejo de Peticiones HTTP

`@RestController`: Indica que la clase es un controlador REST donde los métodos devuelven datos en JSON directo en el cuerpo de respuesta.

`@RequestMapping("/ruta")`: Define la URL base o prefijo para los endpoints expuestos dentro del controlador.

`@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping`: Mapean métodos a verbos HTTP específicos para operaciones CRUD.

`@PathVariable`: Extrae valores dinámicos incluidos en la propia ruta del endpoint (ej. `/pets/{petId}`).

`@RequestParam`: Captura parámetros pasados por la *query string* de la URL (ej. `?userId=4`).

`@RequestBody`: Deserializa automáticamente el cuerpo JSON de la petición entrante hacia un objeto/DTO en Java.

`@Valid`: Desencadena la validación sintáctica de los atributos del objeto mapeado con la petición.

`@ResponseStatus(HttpStatus.X)`: Especifica el código de respuesta HTTP por defecto que retornará el método o la excepción personalizada.

`ResponseEntity<T>`: Permite construir la respuesta HTTP configurando dinámicamente el código de estado, cabeceras y cuerpo.

`@RestControllerAdvice` / `@ExceptionHandler`: Capturan excepciones globales en la aplicación para retornar respuestas JSON estructuradas (ej. `ErrorMessage`).

`SecurityConfiguration` (`.requestMatchers(...)`): Configura reglas de acceso indicando qué roles/autoridades pueden acceder a cada endpoint.

1.2. Servicios y Capa de Negocio

`@Service`: Marca la clase como un componente de lógica de negocio (Bean gestionado por IoC).

`@Autowired`: Inyección automática de dependencias en constructores o atributos.

`@Transactional`: Garantiza la atomicidad de las operaciones de BD. Permite definir `rollbackFor = Exception.class` o `readOnly = true`.

1.3. Mapeo Objeto-Relacional (JPA) y Validación

`@Entity` y `@Table(name = "...")`: Define una clase Java como entidad persistente mapeada a una tabla relacional.

`@Id` y `@GeneratedValue`: Especifican la clave primaria de la entidad y su estrategia de generación automática de valores.

`@Column(name = "...", unique = true)`: Personaliza el nombre de la columna y establece restricciones a nivel de base de datos.

`@Embedded` / `@Embeddable`: Permite incrustar un *Value Object* (sin ID propio) dentro de la tabla de la entidad contenedora.

`@ManyToOne` / `@OneToMany` / `@ManyToMany` / `@OneToOne`: Modelan las asociaciones y cardinalidades entre entidades.

`mappedBy`: Atributo en asociaciones bidireccionales que indica el lado no propietario de la relación (el que no almacena la FK).

Validaciones JSR-303: `@NotNull` (no nulo), `@NotEmpty` (no nulo y tamaño > 0), `@NotBlank` (cadenas con caracteres visibles), `@Size(min=X, max=Y)` (rango de longitud), `@Min(X)` / `@Max(Y)` (rangos numéricos).

`@DateTimeFormat(pattern = "dd/MM/yyyy")`: Formatea el parseo de fechas en atributos tipo `LocalDate`.

1.4. Repositorios y JPQL

`CrudRepository<T, ID>`: Interfaz que ofrece métodos CRUD estándar (`save`, `findById`, `findAll`, `deleteById`).

`@Query`: Permite escribir consultas JPQL personalizadas para operaciones más avanzadas.

2. ELEMENTOS DE CÓDIGO EN FRONTEND (REACT)

`useState`: Hook que permite declarar y actualizar variables de estado interno dentro de un componente funcional.

`useEffect`: Hook que ejecuta efectos secundarios tras el renderizado (como realizar peticiones HTTP para carga de datos).

`fetch('/api/v1/...')` / `axios`: Cliente HTTP para realizar peticiones asíncronas al backend REST.

`<Routes>` / `<Route>` (React Router): Gestiona la navegación declarativa y renderizado condicional en la Single Page Application (SPA).

`Custom Hooks`: Funciones reutilizables que encapsulan la lógica de estado y llamadas a API, separando la lógica del componente visual.

3. PATRONES RECURRENTES EN LOS EXÁMENES DE PRÁCTICAS (CONTROLCHECK 2)

TEST	OBJETIVO PRINCIPAL	ELEMENTOS DE CÓDIGO IMPRESCINDIBLES
Test 1	**Modelado JPA y Repositorios**	`@Entity`, `@Id`, validaciones JSR-303 (`@NotNull`, `@NotBlank`, `@Size`, `@DateTimeFormat`), mapeos de relaciones (`@ManyToOne`, `@ManyToMany`) e interfaz que extiende `CrudRepository<Entidad, Integer>`.
Test 2	**Script SQL** (`data.sql`)	Sentencias `INSERT INTO` respetando el orden estricto de dependencias por clave ajena (FK) y tablas intermedias de relaciones N-A-N.
Test 3	**Controlador REST, DTO y Vista React**	**Backend:** `@RestController`, `@GetMapping`, creación de DTO con mapeo, manejo de 404 (`ResourceNotFoundException`) y ajuste en `SecurityConfiguration`. **Frontend:** Petición con `fetch` dentro de `useEffect`, manipulación de estado y renderizado de un <h1> y una tabla Reactstrap (<Table>).
Test 4	**Consulta JPQL Compleja**	`@Query` haciendo uso de `JOIN`, filtros de rango (`BETWEEN`, `IN`), agrupaciones (`GROUP BY`), condiciones sobre agregados (`HAVING COUNT(...) >= :param`) y ordenación (`ORDER BY ... DESC`).
Test 5	**Pruebas Unitarias de Algoritmos**	Métodos con la anotación `@Test` de JUnit 5 que utilizan `Assertions.assertThrows(Exception.class, () -> ...)` y `Assertions.assertDoesNotThrow(...)` sin cargar contexto de Spring ni Mocks.

RECOMENDACIONES GENERALES PARA LA RESOLUCIÓN DEL EXAMEN

- **Orden de Ejecución SQL:** En el Test 2, inserta primero las entidades independientes antes de las entidades dependientes o relaciones ManyToMany.
- **Seguridad en Backend:** Si añades un nuevo endpoint en el Test 3, no olvides otorgar permisos explícitos en `SecurityConfiguration` o recibirás un código `403 Forbidden` en React.
- **Pruebas Aisladas (Test 5):** No utilices `@SpringBootTest` ni `@MockBean` en el Test 5 a menos que sea explícitamente requerido. Prueba el algoritmo directamente instanciando las clases de dominio.